ReasonLink: P1 Infrastructure and Baselines

Vayun Gupta (vg2684), Aaron Mano Cherian (amc2535)
COMS 6998: AI-Native Computing | Fall 2026
P1 Milestone | Due October 19, 2026, 11:59 PM ET

Repository: https://github.com/AaronCherian404/ReasonLink Results below are from commit `30372c8`. The repository's `PROGRESS.md` logs every step, decision, and reproduction command.

I. Summary and Platform Change

We could not obtain the VCU118 that the Chipyard/FireSim flow needed, so we moved from Chipyard and Gemmini to ESP [1], the open SoC platform that already integrates NVDLA [2] as a third-party accelerator alongside Ariane [3]. The neural accelerator is now NVDLA *nv_small* (int8, 64 MACs); the symbolic accelerator is still SAT-Accel [4]. ESP's full flow is installed on a departmental server, and a VCU118 will be available later through ESP. The research question, the visual-Sudoku workload, the two baselines, and the four ReasonLink mechanisms are unchanged.

For P1 we built every piece of the pipeline that does not need the board, and checked each against a reference (Table I). The main gap is that SAT-Accel itself cannot run yet; a MiniSat stand-in consumes SAT-Accel's exact memory image instead (Sec. II-E).

TABLE I
Component status (October 16).

Component	Status	Evidence
ESP toolchain on server	Ready	ESP 2026.1.0 pinned; Vivado, ModelSim, RISC-V GCC present
NVDLA compile flow	Working	ESP's loadable reproduced byte for byte
int8 LeNet classifier	Working (emulated)	99.20% MNIST; emulator matches NVDLA reference
Sudoku workload	Working	1,000 puzzles + NVDLA output trace
SAT-Accel host image	Working	C port byte-exact vs. SAT-Accel, 13/13
SAT solving	Stand-in	MiniSat on SAT-Accel's image; SAT-Accel blocked
Serial / double-buffered baselines	In progress	Due Oct 19 (Sec. IV)
NVDLA, SAT-Accel on ESP SoC	Not started	After P1

II. Setup by Component

A. Repository

All code is in one repository: `workload/` (CNF encoder, dataset, classifier, NVDLA compile script), `handoff/` (SAT-Accel image builder, solver backend, tests), `models/` (retrained weights), and `results/p1/` (machine-readable outputs). SAT-Accel, MiniSat, and Xilinx's open-source `ap_int` headers are git submodules pinned to fixed commits. Local work runs in WSL Ubuntu 24.04 with a pinned Python environment (`env/setup_wsl.sh`).

B. ESP Platform (server)

The server runs RHEL 8.10 (ESP's primary platform) with 16 cores and 62 GB RAM, and already provides ESP's required tools: Vivado 2023.2, ModelSim DE 2023.2, Xcelium 23.09, Vitis HLS 2023.2, and RISC-V GCC 9.2, so ESP's 1–2 h toolchain build is unnecessary. ESP is pinned to release 2026.1.0 (`a45f2bb`); only the NVDLA software submodule is fetched so far. The server lacks Vitis `v++` and XRT, which matters for SAT-Accel (Sec. II-E).

C. NVDLA Classifier

We use the LeNet [5] that ESP ships for its NVDLA demo, so our classifier is the network ESP already runs on NVDLA.

- *Compiler.* Recompiling ESP's model with the prebuilt `nvdla_compiler` reproduces ESP's shipped loadable byte for byte with `--quantizationMode per-filter --informat nchw`; the compiler is deterministic, and no other flag combination matched.
- *int8 emulator.* A Python model of the int8 pipeline (the runtime's input mapping, per-filter weight scales, calibrated requantization) reproduces ESP's published NVDLA output for its test image exactly. It generates the per-cell scores that stand in for NVDLA's output buffer.
- *Weights.* ESP's demo weights reach only 74.5% on MNIST. They were trained with mean-image subtraction and a 1/80 input scale that neither the deploy model nor the NVDLA int8 input path applies. We retrained the same architecture on the input NVDLA actually receives and wrote a new calibration table (Table II). The retrained model compiles with ESP's flags to a 445,736-byte loadable.

TABLE II
LeNet accuracy on the MNIST test set (10,000 images).

Weights	float	int8 (emulated)
ESP demo, under NVDLA preprocessing	74.5%	74.5%
Retrained (ours)	99.17%	99.20%

D. Workload

Sudoku uses the standard encoding with variable $81r + 9c + d$: 729 variables and 11,988 base clauses, identical for

every puzzle; predicted givens are added as unit clauses. We generated 1,000 puzzles with unique solutions and 24–40 clues (mean 32.1), in the style of visual Sudoku [6]. Each given cell holds an MNIST *test* image of its digit. The classifier's int8 logits for all puzzles are stored as a binary trace, the input to every interface.

E. SAT Solver

Host conversion. SAT-Accel's host code builds a device memory image from the CNF: paged clause and occurrence stores, 64-byte per-variable metadata, and a decision stack. We ported this conversion to standalone C, since it is exactly the work the serial baseline must time. A golden test compiles SAT-Accel's own unmodified parser next to our port and compares every region byte for byte; all 13 instances match: four Sudoku instances (one of them UNSAT) and nine of SAT-Accel's benchmarks, up to 758 variables and 47,820 clauses.

Solving. SAT-Accel's simulation flow requires Vitis v++ and XRT, which the server lacks and which need root to install. For P1 a MiniSat [7] backend reads the clauses back out of the SAT-Accel image, as the device's copy-in would, and solves them. A PySAT [8] run serves as the software reference. Later, the real accelerator replaces the backend behind the same interface.

III. Results So Far

Functional pipeline. On all 1,000 puzzles, the C path (trace → argmax → SAT-Accel image → MiniSat) agrees with the PySAT reference on SAT/UNSAT for every puzzle. It returns the identical grid for 992; the other eight have several solutions because of one misread given, and we verified that both grids satisfy every clause. Table III summarizes accuracy.

TABLE III
Visual Sudoku, 1,000 puzzles.

Metric	Value
Given-cell accuracy (int8)	99.26%
Puzzles solved correctly	79.0%
UNSAT from misread givens	19.2%
SAT but wrong	1.8%
Solver work per puzzle (decisions / conflicts)	1.2 / 0.16

Handoff size. Table IV gives the first baseline measurement: bytes per puzzle on the native SAT-Accel path. The neural stage produces 1.3 KB of logits that encode 36 givens. SAT-Accel's host flow nonetheless rebuilds a 632 KB image, almost all of it the unchanging base formula, and migrates its full 10.5 MB of allocated buffers on every instance. Meanwhile the solver needs about one decision per puzzle. Both observations are consistent with H1 and motivate delta transfers.

Limitations. Solver time in P1 is MiniSat CPU time, not SAT-Accel cycles. NVDLA has not yet executed on hardware: the loadable is compiled, and its outputs are emulated. NVDLA's real output layout in memory is still to be confirmed on ESP. Sudoku with $\geq$24 clues is nearly propagation-only, so the H3 crossover will need harder instances.

TABLE IV
Data per puzzle on the native SAT-Accel path (puzzle 0, 36 givens).

Quantity	Bytes
NVDLA output (int8 logits, 81×16 B)	1,296
SAT-Accel image, bytes in use	632,436
SAT-Accel buffers migrated per instance	10,536,436

IV. Next Steps

Target	Task
Oct 19 (P1)	Serial and double-buffered baselines end to end on all 1,000 puzzles, with per-puzzle T_{conv}, T_{xfer}, T_{sync}, T_{sat} cycle counts; cross-compile both for RISC-V (Ariane).
Nov 2 (P2)	Run SAT-Accel functionally (its HLS kernels as threads, or XRT on a machine with access); first ESP SoC with Ariane and NVDLA in simulation; ReasonLink prototype with literal stream and on-chip FIFO; pilot H1–H2 breakdown.
After P2	Delta transfers and overlap; SAT-Accel on ESP; VCU118 runs when the board is available.

Team contributions. [Who did what for P1.]

References

[1] P. Mantovani, D. Giri, G. Di Guglielmo, L. Piccolboni, J. Zuckerman, E. G. Cota, M. Petracca, C. Pilato, and L. P. Carloni, "Agile SoC development with open ESP," in *Proceedings of the 39th International Conference on Computer-Aided Design (ICCAD)*, 2020. [Online]. Available: https://www.esp.cs.columbia.edu

[2] NVIDIA, "NVDLA: NVIDIA deep learning accelerator," http://nvdla.org, 2017.

[3] D. Giri, K.-L. Chiu, G. Eichler, P. Mantovani, N. Chandramoorthy, and L. P. Carloni, "Ariane + NVDLA: Seamless third-party IP integration with ESP," in *Fourth Workshop on Computer Architecture Research with RISC-V (CARRV)*, 2020.

[4] M. Lo, M.-C. F. Chang, and J. Cong, "SAT-Accel: A modern SAT solver on a FPGA," in *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, 2025, pp. 234–246. [Online]. Available: https://github.com/milo168/FPGA25_SAT_Accel

[5] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[6] P.-W. Wang, P. L. Donti, B. Wilder, and J. Z. Kolter, "SATNet: Bridging deep learning and logical reasoning using a differentiable satisfiability solver," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019.

[7] N. Eén and N. Sörensson, "An extensible SAT-solver," in *Theory and Applications of Satisfiability Testing (SAT)*, ser. LNCS, vol. 2919, 2003, pp. 502–518.

[8] A. Ignatiev, A. Morgado, and J. Marques-Silva, "PySAT: A Python toolkit for prototyping with SAT oracles," in *Theory and Applications of Satisfiability Testing (SAT)*, 2018, pp. 428–437.